Software Requirements Engineering

Lecture 7: Software Requirements Specification

Review

1. Requirements analysis using use cases and user stories
2. Software requirements priorities

Outline

1. Setting requirement priorities
2. Software requirement specification

Setting Requirement Priorities

- Every project with resource limitations needs to define the relative priorities of the requested product capabilities.
- Prioritization helps reveal competing goals, resolve conflicts, plan for staged or incremental deliveries, control scope creep, and make the necessary trade-off decisions.

Setting Requirement Priorities

The successful prioritization requires an understanding of six issues:

1. The needs of the customers
2. The relative importance of requirements to the customers
3. The timing at which capabilities need to be delivered
4. Requirements that serve as predecessors for other requirements and other relationships among requirements
5. Which requirements must be implemented as a group
6. The cost to satisfy each requirement

Setting Requirement Priorities

- Developing user requirements helps with requirements prioritization.
- The highest-priority functional requirements are those that originate in the top-priority user requirements.
- A use case or user story could be of high priority for several reasons:
  - It describes part of a core business process that the system enables.
  - Many users will use it frequently.
  - A favored user class requested it.
  - It’s required for regulatory compliance.
  - Other system functions depend on its presence

Requirements Specification

- Requirements specification is the process of writing down the user and system requirements in a requirements document.
- User requirements have to be understandable by end-users and customers who do not have a technical background.
- System requirements are more detailed requirements and may include more technical information.
- The requirements may be part of a contract for the system development
  - It is therefore important that these are as complete as possible.

Ways of writing a system requirements specification

Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract

Requirements and design

- In principle, requirements should state what the system should do and the design should describe how it does this.
- In practice, requirements and design are inseparable
  - A system architecture may be designed to structure the requirements;
  - The system may inter-operate with other systems that generate design requirements;
  - The use of a specific architecture to satisfy non-functional requirements may be a domain requirement.
  - This may be the consequence of a regulatory requirement.

Natural language specification

- Requirements are written as natural language sentences supplemented by diagrams and tables.
- Used for writing requirements because it is expressive, intuitive and universal. This means that the requirements can be understood by users and customers.

Guidelines for writing requirements

- Invent a standard format and use it for all requirements.
- Use language in a consistent way. Use shall for mandatory requirements, should for desirable requirements.
- Use text highlighting to identify key parts of the requirement.
- Avoid the use of computer jargon.
- Include an explanation (rationale) of why a requirement is necessary.

Problems with natural language

- Lack of clarity
  - Precision is difficult without making the document difficult to read.
- Requirements confusion
  - Functional and non-functional requirements tend to be mixed-up.
- Requirements amalgamation
  - Several different requirements may be expressed together.

Example

- 3.2 The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes. (Changes in blood sugar are relatively slow so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.)

- 3.6 The system shall run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. (A self-test routine can discover hardware and software problems and alert the user to the fact the normal operation may be impossible.)

Structured specifications

- Structured natural language is a way of writing system requirements:
  - The freedom of the requirements writer is limited
  - all requirements are written in a standard way
- This works well for some types of requirements e.g. requirements for embedded control system but is sometimes too rigid for writing business system requirements.

Structured Specifications

Essential information:

- Definition of the function or entity.
- Description of inputs and where they come from.
- Description of outputs and where they go to.
- Information about the information needed for the computation and other entities used.
- Description of the action to be taken.
- Pre and post conditions (if appropriate).
- The side effects (if any) of the function.

Form-based specifications

- Definition of the function or entity.
- Description of inputs and where they come from.
- Description of outputs and where they go to.
- Information about the information needed for the computation and other entities used.
- Description of the action to be taken.
- Pre and post conditions (if appropriate).
- The side effects (if any) of the function.

Insulin Pump/Control Software/SRS/3.3.2	
Function	Compute insulin dose: safe sugar level.
Description	
Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.	
Inputs	Current sugar reading (r2); the previous two readings (r0 and r1).
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose—the dose in insulin to be delivered.
Destination	Main control loop.

Form-based specifications

Action
CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered.
Requirements
Two previous readings so that the rate of change of sugar level can be computed.
Pre-condition
The insulin reservoir contains at least the maximum allowed single dose of insulin.
Post-condition r0 is replaced by r1 then r1 is replaced by r2.
Side effects None.

Tabular specification

- Used to supplement natural language.
- Particularly useful when you have to define a number of possible alternative courses of action.
- For example, the insulin pump systems bases its computations on the rate of change of blood sugar level and the tabular specification explains how to calculate the insulin requirement for different scenarios.

Tabular specification - example

- Tabular specification of computation for an insulin pump.

Condition	Action
Sugar level falling (r2 < r1)	CompDose = 0
Sugar level stable (r2 = r1)	CompDose = 0
Sugar level increasing and rate of increase decreasing ((r2 – r1) < (r1 – r0))	CompDose = 0
Sugar level increasing and rate of increase stable or increasing ((r2 – r1) ≥ (r1 – r0))	CompDose = round ((r2 – r1)/4) If rounded result = 0 then CompDose = MinimumDose

Guidelines for writing requirements

- Two important goals of writing requirements are that:
  - Anyone who reads the requirement comes to the same interpretation as any other reader.
  - Each reader's interpretation matches what the author intended to communicate.

There is no formulaic way to write excellent requirements; the best teachers are experience and feedback from the recipients of your requirements.

Characteristics of requirement statements

- Complete
- Correct
- Feasible
- Necessary
- Prioritized
- Unambiguous
- Verifiable

Characteristics of requirement statements

- Complete
  - Each requirement must contain all the information necessary for the reader to understand it.
  - In the case of functional requirements, this means providing the information the developer needs to be able to implement it correctly.
  - If you know you're lacking certain information, use TBD (to be determined).
  - Resolve all TBDs in each portion of the requirements before the developers proceed with construction of that portion.

Characteristics of requirement statements

- Correct
  - Each requirement must accurately describe a capability that will meet some stakeholder's need and must clearly describe the functionality to be built.
  - You'll have to go to the source of the requirement to check its correctness.
  - A low-level requirement that conflicts with its parent is not correct.

Characteristics of requirement statements

- Feasible
  - It must be possible to implement each requirement within the known capabilities and limitations of the system and its operating environment, as well as within project constraints of time, budget, and staff.
  - If a requirement needs to be cut because it is not be feasible, understand the impact on the project vision and scope.

Characteristics of requirement statements

- Necessary
  - Each requirement should describe a capability that provides stakeholders with the anticipated business value, differentiates the product in the marketplace, or is required for conformance to an external standard, policy, or regulation.
  - Every requirement should originate from a source that has the authority to provide requirements.
  - You should be able to relate each requirement to a business objective that clearly indicates why it’s necessary.

Characteristics of requirement statements

- Prioritized
  - Prioritize business requirements according to which are most important to achieving the desired value.
  - Assign an implementation priority to each functional requirement, user requirement, use case flow, or feature to indicate how essential it is to a particular product release.

Characteristics of requirement statements

- Unambiguous
  - Natural language is prone to two types of ambiguity.
    - Can think of more than one way to interpret a given requirement
    - Different people read the requirement and come up with different interpretations of it
  - Inspections are a good way to spot ambiguities.
  - You'll never remove all the ambiguity from requirements—that's the nature of human language.

Characteristics of requirement statements

- Verifiable
  - If a requirement isn't verifiable, deciding whether it was correctly implemented becomes a matter of opinion, not objective analysis.
  - Requirements that are incomplete, inconsistent, infeasible, or ambiguous are also unverifiable.
  - Testers are good at examining requirements for verifiability.

Labeling Requirements

- Every requirement needs a unique and persistent identifier.
- This allows you to refer to specific requirements in a change request, modification history, cross-reference, or requirements traceability matrix.
- It also enables reusing the requirements in multiple projects.

- Sequence number
  - Use case 6: UC-6
  - Functional requirement 28: FR-28
  - This simple numbering approach
  - Doesn’t provide any logical or hierarchical grouping of related requirements

Labeling Requirements

- Hierarchical numbering
  - This method is simple, compact, and familiar.
  - If the functional requirements appear in section 3.2 of the SRS, they will all have labels that begin with 3.2.
  - More digits indicate a more detailed, lower-level requirement
  - E.g. 3.2.4.1
  - Section 3.5 – Editor Functions : ED-1, ED-2

  - The labels can grow to many digits in even a medium-sized SRS.
  - Numeric labels tell you nothing about the intent of a requirement.
  - Delete, insert, merge, or move whole sections, and a lot of labels change. These changes disrupt any references to those requirements elsewhere in the system.

Labeling Requirements

- Hierarchical textual tags
  - Consultant Tom Gilb (1988) suggests a text-based hierarchical tagging scheme for labeling individual requirements.
  - Hierarchical textual tags are structured, meaningful, and unaffected by adding, deleting, or moving other requirements.

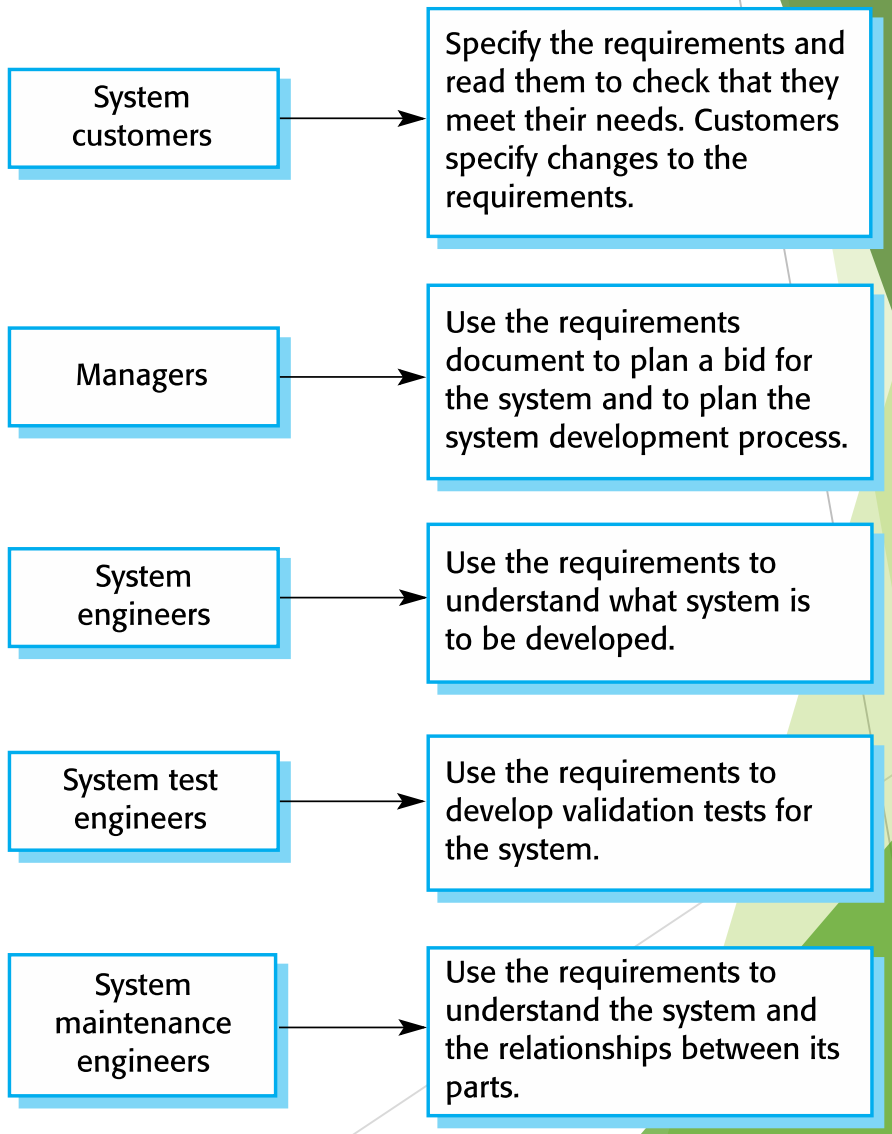
Product:	Ordering products from the website
.Cart	The website shall use a shopping cart to contain products a Customer selects to purchase.
.Discount	The shopping cart shall provide one discount code field. Each discount code provides either a specific discount percentage or a fixed dollar discount amount on specific items in the cart.
.Error	If the Customer enters an invalid discount code, the website shall display an error message.
.Shipping	The shopping cart shall add a shipping charge if a Customer orders a physical product that must be mailed.

Software Requirements Specification

- The software requirements document (also called software requirements specification or SRS) is an official statement of what the system developers should implement.
- The SRS should include both the user requirements for a system and a detailed specification of the system requirements.
- SRS is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it.

Software Requirements Specification

- The SRS has a diverse set of users and they need different levels of details.
  - Software development teams need to know what to build.
  - Training personnel use the SRS and user documentation to develop educational materials.
  - Testers use it to develop requirements-based tests, test plans, and test procedures
  - …



1. **Introduction**
   - 1.1 Purpose
   - 1.2 Document conventions
   - 1.3 Project scope
   - 1.4 References
2. **Overall description**
   - 2.1 Product perspective
   - 2.2 User classes and characteristics
   - 2.3 Operating environment
   - 2.4 Design and implementation constraints
   - 2.5 Assumptions and dependencies
3. **System features**
   - 3.x System feature X
     - 3.x.1 Description
     - 3.x.2 Functional requirements
4. **Data requirements**
   - 4.1 Logical data model
   - 4.2 Data dictionary
   - 4.3 Reports
   - 4.4 Data acquisition, integrity, retention, and disposal
5. **External interface requirements**
   - 5.1 User interfaces
   - 5.2 Software interfaces
   - 5.3 Hardware interfaces
   - 5.4 Communications interfaces
6. **Quality attributes**
   - 6.1 Usability
   - 6.2 Performance
   - 6.3 Security
   - 6.4 Safety
   - 6.x [others]
7. **Internationalization and localization requirements**
8. **Other requirements**

Appendix A: Glossary

Appendix B: Analysis models

An example template of the SRS

Software Requirements Specification

- The SRS states the functions and capabilities that a software system must provide, its characteristics, and the constraints that it must respect.
- But it should not contain design, construction, testing or project management details.
- The information that is included in a requirements document depends on the type of software being developed and the approach to development that is to be used.

- **Important**: A single requirements deliverable often cannot meet the needs of all audiences.

Software Requirements Specification

The readability suggestions:

- Use an appropriate template to organize all the necessary information.
- Label and style sections, subsections, and individual requirements consistently.
- Use visual emphasis consistently and judiciously. Create a table of contents to help readers find the information they need.
- Number all figures and tables.
- If you are using documents, use hyperlinks, cross-reference facility to let reader navigate to related information
- Include visual representations of information when possible to facilitate understanding.
- Uses a consistent vocabulary and layout.

Characteristics of requirements collections

- Complete: No requirement or necessary information should be absent. In practice, you'll never document every single requirement for any system. Any specification that contains TBDs is incomplete.
- Consistent: Consistent requirements don't conflict with other requirements of the same type or with higher-level business, user, or system requirements.
- Modifiable: You can always rewrite a requirement, but you should maintain a history of changes made to each requirement, especially after they are baselined. You also need to know about connections and dependencies between requirements.
- Traceable: A traceable requirement can be linked both backward to its origin and forward to derivedrequirements, design elements, code that implements it, and tests that verify its implementation.

- You're never going to create a perfect specification in which all requirements demonstrate all of these ideal attributes.
- But you need to keep them in mind.

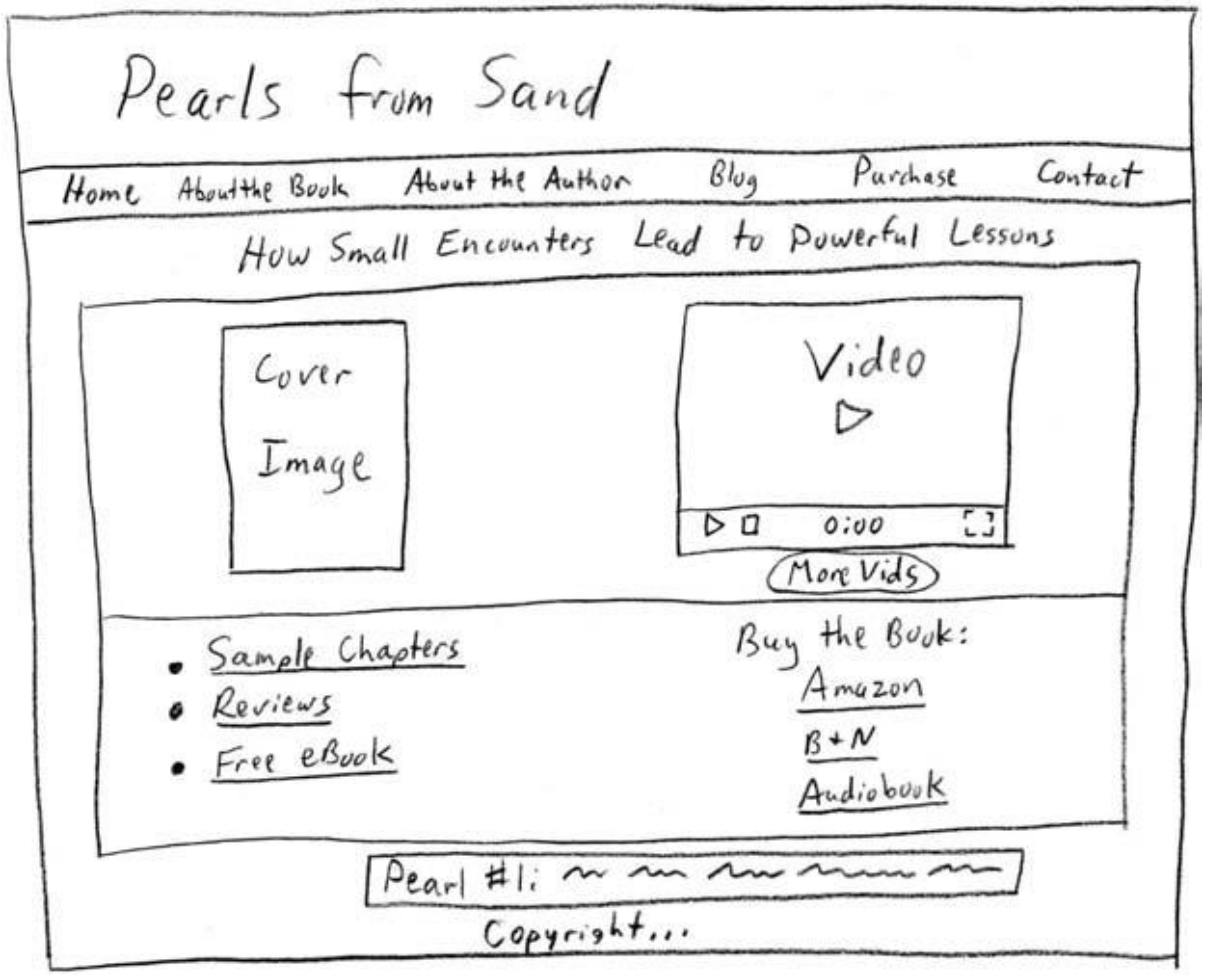
Dealing with incompleteness

- Use TBD (to be determined) to flag the knowledge gaps.
- Plan to resolve all TBDs before implementing a set of requirements.
- Record TBDs and other requirements questions in an issues list.
- TBDs won’t resolve themselves. Number the TBDs, record who is responsible for resolving each issue and by when, review their status at regular checkpoints, and track them to closure.

User interfaces and the SRS

- Incorporating user interface designs in the SRS has both benefits and drawbacks.
- These are powerful techniques for eliciting and validating requirements.
- Especially useful when certain functionality with specific UI controls and screen layouts need to be implemented.
- Screen layouts should not replace written user and functional requirements.
- Can think about having a separate user interface specification.

User interfaces and the SRS



In the next lecture…

- We will talk about requirements specification and give you some example template.

Question session